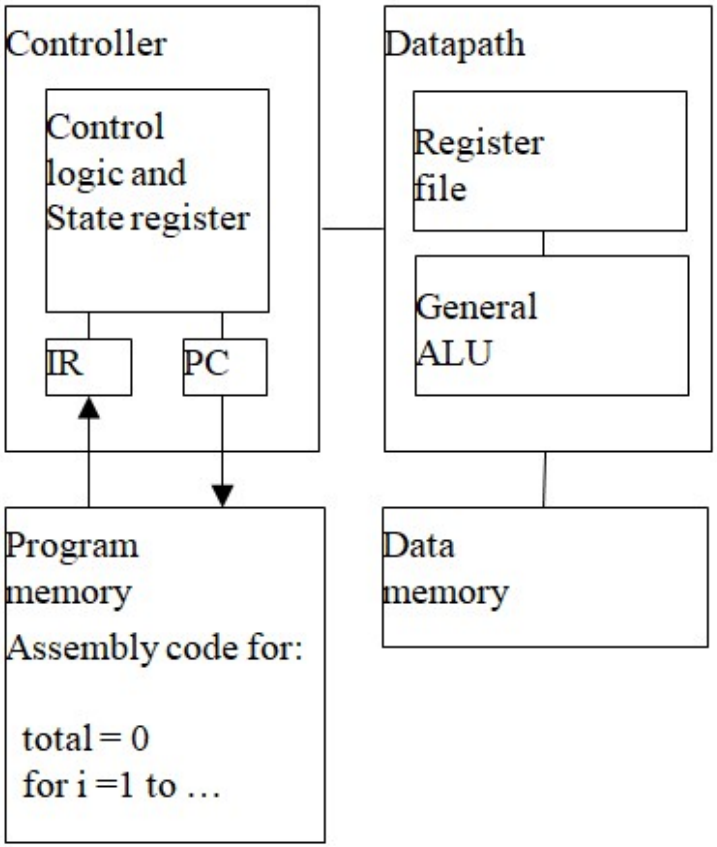
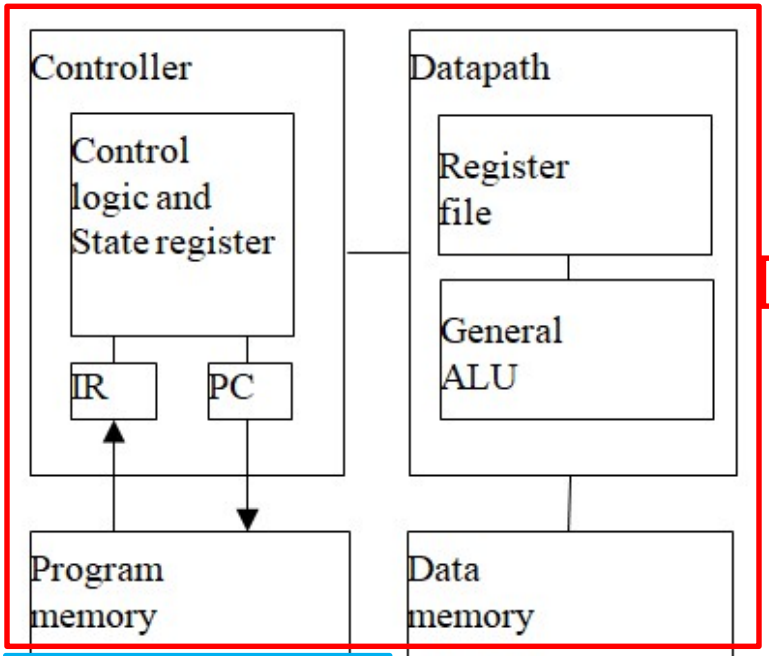
Microcontrollers

Rex Joseph
Department of Electrical and
Electronics Engineering

Recap: A microprocessor



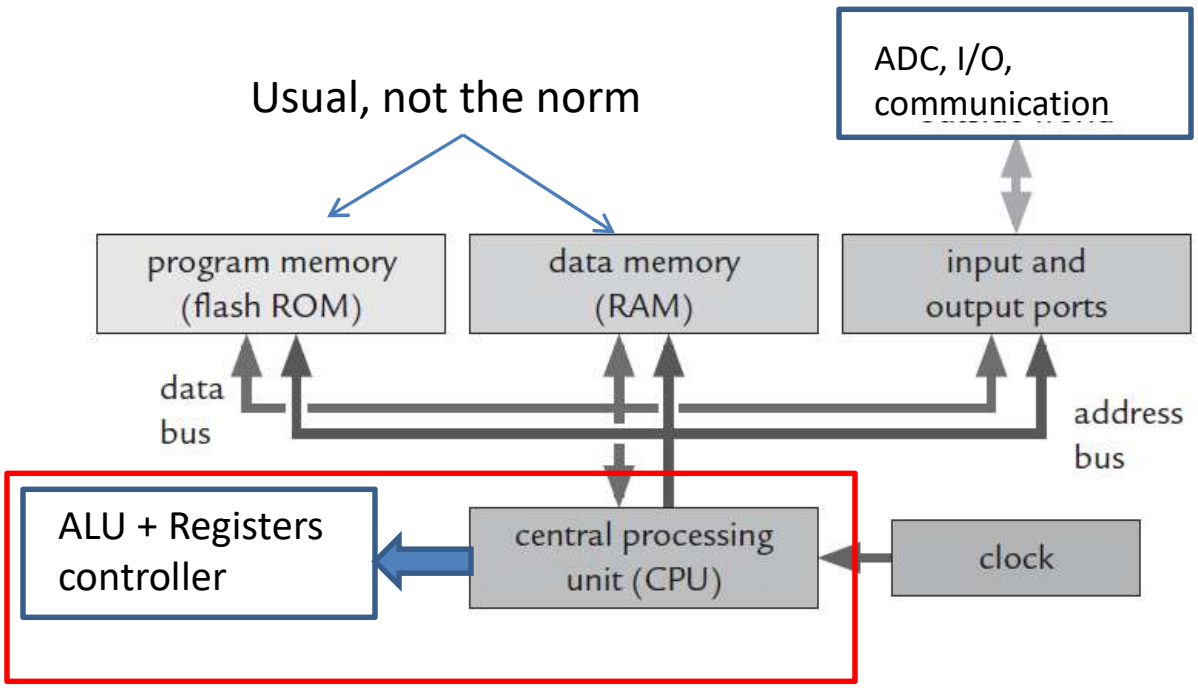
Recap: A microprocessor



Architecture and organization. The processor **hardware**

Program code – **Software**
Firmly coupled to hardware resources in embedded systems

A typical microcontroller



And the control bus

Getting started with code – Configuration/initialization

The uC has several functions and interface lines

Not all of the signals can be brought out to individual ‘pins’ directly mapped.

Getting started with code – Configuration/initialization

The uC has several functions and interface lines

Not all of the signals can be brought out to individual ‘pins’ directly mapped.

Need to multiplex – so first task on port pins is to decide functionality
Will have defaults, and may not need to be modified if correct

In addition, clock, watchdog,…….

Central processing unit:

Arithmetic logic unit (ALU), which performs computation.

Registers needed for the basic operation of the CPU, such as the **program counter** (PC), **stack pointer** (SP), and **status register** (SR).

Further registers to hold temporary results.

Instruction decoder and other logic to control the CPU, handle **resets**, and **interrupts**, and so on.

Other resources:

Memory for the program:
Nonvolatile (read-only memory, ROM), meaning that it retains its contents when power is removed.

Memory for data: Known as random-access memory (RAM) and usually volatile.

Input and output ports: To provide digital communication with the outside world.
Address and data buses: To link these subsystems to transfer data and instructions.
Clock: To keep the whole system synchronized. It may be generated internally or obtained from a crystal or external source; modern MCUs offer considerable choice of clocks.

Other resources:

It is unlikely that any processor would lack any of these features, although their implementation may differ substantially.

The differences between devices comes from the range of ***peripherals*** included.

Peripheral functionality needed completely separate pieces of equipment long ago, but as technology improved, they could be included on the same **printed circuit board** as the processor.

Now most peripherals are on the same **integrated circuit** as the processor and the **nomenclature is no longer appropriate**, but it has stuck.

Other resources:

Timers: Most microcontrollers have at least one timer because of the wide range of functions they provide

Watchdog timer: This is a safety feature, which resets the processor if the program becomes stuck in an infinite loop.

Communication interfaces: A wide choice of interfaces is available to exchange information with another IC or system.
They include serial peripheral interface (SPI), inter-integrated circuit (I^2C or IIC), asynchronous (such as RS-232), universal serial bus (USB), controller area network (CAN), ethernet, and many others.

Nonvolatile memory for data: This is used to store data whose value must be retained when power is removed. Serial numbers for identification and network addresses for instance.

Other resources:

Analog-to-digital converter: This is very common because so many quantities in the real world vary continuously.

Digital-to-analog converter: This is much less common, because most analog outputs can be simulated using PWM.
An important exception used to be sound, but even here, the use of PWM is growing in what are called ***class D amplifiers***.

Other extra resources:

Real-time clock: These are needed in applications that must track the time of day. Clocks are obvious examples but data loggers are also an important case.

Monitor, background debugger, and embedded emulator: These are used to download the program into the MCU and communicate with a desktop computer during development.

Additionally text editor, software **tool-chain** etc.
Forms part of the Integrated Development Environment (**IDE**)

Memory:

Memory lies at the heart of any computer. Each location can typically store 1 byte (8 bits or 1B) of data and is often called a *register,* although this term is sometimes reserved for memories within the CPU. Many processors have wider registers. 16 bit, 32 bit, 64 bit

Memory is linked to the CPU by **buses** for **data**, **address**, and **control** Buses are shared sets of wires

Access to the bus must be controlled, to define when data are valid and ensure that two components do not try to write/read at the same time. This is the job of the control bus.

The number of wires in a bus defines its width, and processors are commonly characterized by width of the data bus, which is generally the same as the size of data that can be processed by the CPU.

Memory:

For example, an 8-bit processor has a data bus of this width and most operations in its CPU use 8 bits (although there may be further instructions to handle 16 bits as well).

The address bus need not have the same width as the data bus and is often wider; an 8-bit address bus would only carry $2^8 = 256$ distinct addresses, which is too small to be useful.

Many 8-bit processors have 16-bit address buses, for instance. (How many addressable locations ?)

Addresses are always quoted in *hexadecimal (hex) notation, where each digit has a range* of 0–15.
The values 10–15 are written as a–f or A–F.

Memory:

Hexadecimal values are distinguished by a prefix of “0x” in the C programming language; other notations are required for assembly language.

The reason for using hexadecimal notation is that an 8-bit byte can be written as a two-digit hexadecimal number in the range 0x00–0xFF, corresponding to 0–255 in decimal.

Similarly, a 16-bit address lies in the range 0x0000–0xFFFF.

The four bits that correspond to each hexadecimal digit are known as nibbles. Convenient to represent 8,16,32,64 etc bits.

.

Memory:

The buses in a microprocessor are brought out to pins for access to external memory. Micro-controllers generally have sufficient internal memory, and the address and data buses are not always brought out.

External memory can be added using a separate interface, either serial or parallel.

This is facilitated by the I/O pins that are available.

Volatile and Nonvolatile Memory

Volatile: Loses its contents when power is removed.
It is usually (used to be) called random-access memory or RAM, but the name is misleading because access to most other types of memory is equally random. (Older classification, random and sequential access)

Volatile memory is used for data, and small microcontrollers often have very little RAM, sometimes only a few tens of bytes.
The memory is usually *static RAM,* which means that it retains its data even if the clock is stopped (provided that power is maintained, of course).

A single cell of static RAM needs six transistors. RAM therefore takes up a large area of silicon, which makes it expensive.

Volatile and Nonvolatile Memory

Most memory in a desktop computer is **dynamic** RAM.

This needs only one transistor per cell but must be refreshed regularly to maintain its contents, so it is not used in small microcontrollers.

Nonvolatile: Retains its contents when power is removed and is therefore used for the program and constant data.
It is usually called **read-only memory** or ROM, but again this traditional name has become misleading.

Most modern microcontrollers can write to their nonvolatile memory but it is much slower and more complicated than writing to RAM.

Classification: Write ability and storage permanence

Nonvolatile Memory - Types

Masked ROM: The data are encoded into one of the masks used for photolithography and written into the IC during manufacture.
This memory really is read-only.
It is used for the high-volume production of stable products, because any change to the data requires a new mask to be produced
Some MSP430 devices can be ordered with ROM, shown by a *C in their part number. An example is the* MSP430CG4619.

EPROM (electrically programmable ROM): As its name implies, it can be programmed electrically but not erased. Devices must be exposed to ultraviolet (UV) light for about ten minutes to erase them.
The usual black epoxy encapsulation is opaque, so erasable devices need special packages with **quartz windows**, which are expensive. These were widely used for development before flash memory was widely available.

Nonvolatile Memory - Types

OTP (one-time programmable memory): This is just EPROM in a normal package without a window, which means that it cannot be erased.
Devices with OTP ROM are still widely used and the first family of the MSP430 used this technology.

Flash memory: This can be both programmed and erased electrically and is now by far the most common type of memory.

It has largely superseded **electrically erasable, programmable ROM** (EEPROM).
The practical difference is that individual bytes of EEPROM can be erased but flash can be erased only in **blocks**. Most MSP430 devices use flash memory, shown by an F in the part number.

Nonvolatile Memory - Types

A higher voltage is needed to write to flash memory than is necessary for normal operation.
This had to be supplied externally to early devices but modern components include a charge pump to generate the programming voltage internally.

Flash memory is of course widely used in portable storage devices—memory cards, USB drives, and so on—but the technology is different.

Microcontrollers use NOR flash, which is slower to write but permits random access.
NAND flash is used in bulk storage devices and can be accessed only serially in rows

Memory Organization

Harvard Architecture

The data and program memories are treated as separate systems, each with its own address and data bus.

Many microcontrollers use this architecture, including Microchip PICs, the Intel 8051 and descendents, and the ARM9.

The principal advantage is efficiency.

• It allows simultaneous access to the program and data memories. For instance, the CPU can read an operand from the data memory at the same time as it reads the next instruction from the program memory.

• The two systems can be separately optimized. For example, the PIC16 has a data memory with an 8-bit data bus and a 9-bit address bus.

On the other hand, the program memory is 14 bits wide, so that each word holds a complete instruction, with a 13-bit address bus

Memory Organization

Harvard Architecture

A problem with the Harvard architecture is that constant data (often lookup tables) must be stored in the program memory because it is nonvolatile.

This means that constants cannot be read in the same way as volatile values from the data memory.

Special “table read” instructions must therefore be provided or part of the program memory is mapped into data memory.

Memory Organization

von Neumann Architecture

There is only a single memory system in the von Neumann or **Princeton** architecture.

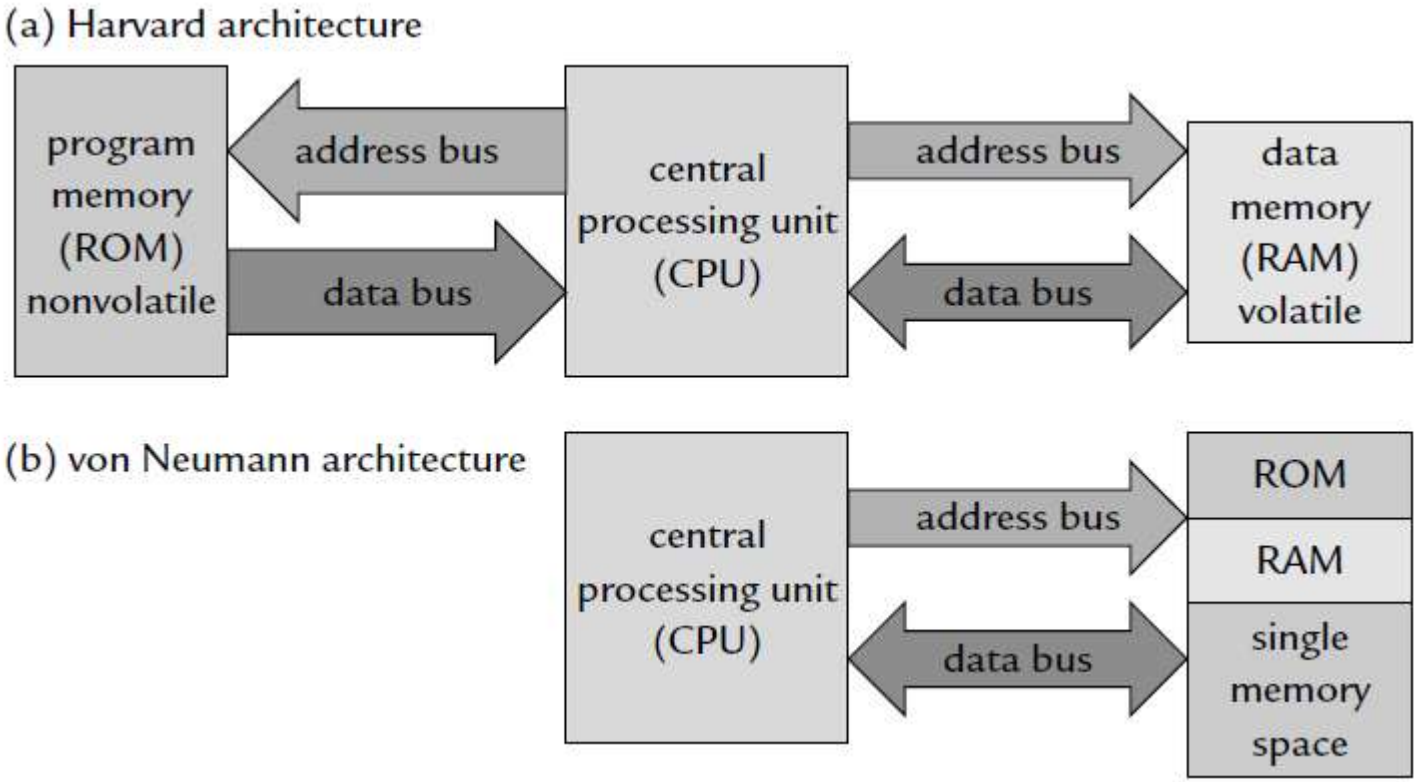
This means that only one set of addresses covers both the volatile and nonvolatile memories.

The memory map, which shows the addresses at which each type of memory is located, becomes particularly important.

The architecture is intrinsically less efficient because several memory cycles may be needed to extract a full instruction from memory. However, the system is simpler and there is no difference between access to constant and variable data.

Microcontrollers with a von Neumann architecture include the **MSP430**, the Freescale HCS08, and the ARM7.

Memory Organization



Software

Machine code: The binary data that the processor itself understands.
Each instruction has a binary value called an *opcode.*
Cannot be directly interpreted by us. Eg. 0x3E7

Some very early computers had to be programmed in machine code, but that was long ago

The contents of memory are shown in the debugger and machine code is included in the "**disassembly**"

Software

Assembly language: Little more than machine code translated into English.

The instructions are written as words called ***mnemonics*** rather than binary values and a **program** called an **assembler** translates the mnemonics into machine code.

It does a little more than direct translation, but not a lot, nothing like a **compiler** for a high-level language.

Software

Assembly language: Little more than machine code translated into English.

The instructions are written as words called ***mnemonics*** rather than binary values and a **program** called an **assembler** translates the mnemonics into machine code.

It does a little more than direct translation, but not a lot, nothing like a **compiler** for a high-level language.

A major disadvantage of assembly language is that it is intimately tied to a processor and is therefore different for each architecture.

Even worse, the detailed usage varies between development environments for the same processor.

Software

Most programming of small microcontrollers was done in assembly language until recently, despite these problems, mainly because compilers for C produced less-efficient code.

Now the compilers are better and modern processors are designed with compilers in mind, so assembly language has been pushed to the fringes. Bitwise operation for instance

The main argument for learning assembly language is for debugging.

To check the operation of the processor, one instruction at a time. (single stepping)

Disassembly is the opposite process to assembly, the translation of machine code to assembly language.

Software

High Level language (Eg. C): The most common choice for small microcontrollers nowadays.

A **compiler** translates high-level language into machine code that the CPU can process.

This brings all the power of a high-level language—data structures, functions, type checking and so on—but can usually be compiled into efficient code.

Compilation used to go through assembly language but this is now less common and the compiler produces machine code directly.

A disassembler must then be used if you wish to review the assembly language.

The MSP430

The MSP430 is a particularly straightforward 16-bit processor with a von Neumann architecture, designed for low-power applications.

The CPU is often described as a reduced instruction set computer (RISC)

Both the address and data buses are 16 bits wide. The registers in the CPU are also all 16 bits wide and can be used interchangeably for either data or addresses.

The MSP430 has 16 registers in its CPU, which enhances efficiency because they can be used for local variables, parameters passed to subroutines, and either addresses or data.

This is a typical feature of a RISC, but unlike a “pure” RISC, it can perform arithmetic **directly** on values in main memory.

uP/uC– Vendors/users

1Altera
2AMD
3Analog Devices
4Apollo
5ARM
6Atmel
7Data General
8Digital Equipment Corporation
9Emotion Engine by Sony & Toshiba
10Elbrus
11EnSilica
12Fairchild Semiconductor
13Freescale Semiconductor (former Motorola)
14Hewlett-Packard
15Hitachi
16Inmos
17IBM
    17.1POWER
    17.2PowerPC-AS
    17.3z/Architecture
18Intel
19Intersil
20ISRO
21Lattice Semiconductor
22MIPS Technologies
23MOS Technology
24National Semiconductor
25NEC
26NVIDIA
27NXP (former Philips Semiconductors)
28OpenCores
29Oracle Corporation (formerly Sun Microsystems)
30RCA
31Renesas Electronics
32Sunway
33Texas Instruments
34VIA
35Western Design Center
36Western Digital
37Western Electric
38Xilinx
39XMOS
40Zilog
…….
……
……

TI Micro-controllers

- **Microcontrollers (MCU) (819)**
- **MSP430 ultra-low-power MCUs (525)**
  - MSP430FRxx FRAM (102)
  - MSP430F5x/6x (200)
  - MSP430F2x/4x (139)
  - MSP430F1x (29)
  - MSP430G2x/i2x (55)
- **MSP432 low power + performance MCUs (2)**
- **Performance MCUs (254)**
  - Real-time Control (94)
    - Piccolo F2802x/3x/5x/6x/7x (43)
    - Delfino F2833x/F2837x (22)
    - Fixed-point F280x/1x (29)
  - Control + Automation (83)
    - F28M3x (12)
    - TM4C12x (71)
  - Safety (77)
    - Hercules RM (41)
    - Hercules TMS570 (33)
    - Hercules TMS470M (3)
- **Wireless MCUs (38)**
  - RF430 (6)
  - CC430 (13)
  - SimpleLink CC1x (2)
  - SimpleLink CC2x (15)
  - SimpleLink CC3x (2)

PES UNIVERSITY ONLINE

The MSP430

"Our 16-bit MSP430™ microcontrollers (MCUs) provide affordable solutions for all applications.

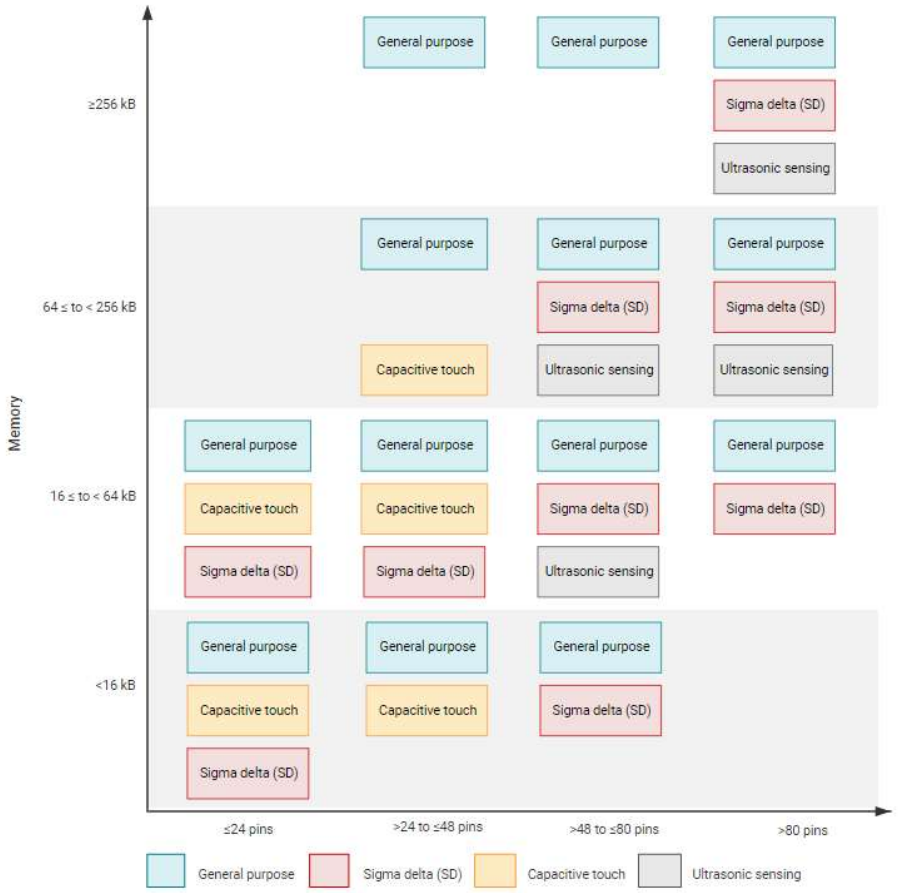
Our leadership in integrated precision analog enables designers to enhance system **performance** and lower **system costs**.

Designers can find a cost-effective MCU within the broad MSP430 portfolio of **over 2000 devices** for virtually any need.

Get started quickly and reduce **time to market** with our simplified **tools**, **software**, and best-in-class support."**

** Texas Instruments

The MSP430



https://www.ti.com/microcontrollers-mcus-processors/msp430-microcontrollers/overview.html

The MSP430

Learn more about the MSP430™ portfolio

Our family of MSP430 MCUs offers a broad and affordable portfolio of products to solve today's increasing and diverse MCU design challenges. This includes flexible, integrated analog components, such as ADCs, DACs, and Op-amps; Integrated LCD controllers for displaying information; and devices with integrated USB2.0 for communicating with a PC or application updates.

MCUs with Precision ADCs (> 16 bits)
MCUs with integrated DACs and Op Amps
MCUs with LCD Controller
MCUs with Integrated USBs

Applications are diverse, and there are MCUs tailored for each class

General purpose, application specific, custom single purpose

The MSP430 Development Ecosystem

The **MSP430G2553 16-bit MCU** has 16KB of flash, 512 bytes of RAM, up to 16-MHz CPU speed, an 8- channel 10-bit ADC, capacitive-touch enabled I/Os, a universal serial communication interface, and more …

Details as the course unfolds!

Can't place it on a bread board and start, need other 'parts'

The **LaunchPad development kit** features an integrated DIP target socket that supports up to 20 pins, allowing MSP430 entry-level MCUs to be plugged into the LaunchPad development kit.

The MSP-EXP430G2ET LaunchPad development kit comes with an **MSP430G2553 MCU by default**. The MSP430G2553 MCU has the most memory available of the compatible entry-level MCUs.

The Arduino Board is another familiar example

Different MCU options

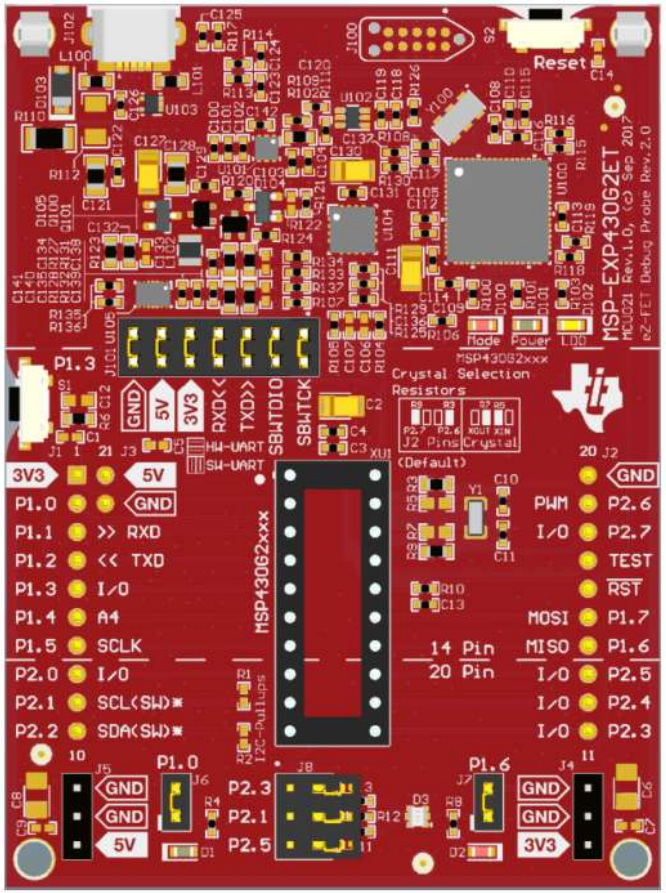
The MSP430 Development Ecosystem

The MSP430G2553 LaunchPad™ Development Kit is an inexpensive and easy-to-use evaluation module (EVM) for the MSP430G2xx entry-level series of microcontrollers (MCUs).

It contains everything needed to start developing on the ultra-low-power MSP430™ microcontroller platform, including an **onboard debug probe** for **programming, debugging and energy measurements.**

The board also features a **push button** and **three LEDs** for creating a simple user interface

First program – Blink LED



The MSP430 Development Ecosystem

Free **software development tools** are also available, such as TI's **Eclipse-based Code Composer Studio™** IDE (CCS) and **IAR Embedded Workbench®** for MSP430 IDE (IAR EW430).

Both of these IDEs support EnergyTrace™ technology for real-time power profiling and debugging when paired with the MSP430G2553 LaunchPad development kit.

More information about the LaunchPad development kit, including documentation and design files, can be found on the MSP430G2553 LaunchPad development kit tool page.

http://www.ti.com/tool/MSP-EXP430G2ET

PES UNIVERSITY ONLINE

The MSP430 Development Ecosystem

Energia IDE vs. Code Composer Studio:

Energia is an Open source and free Environment that enables us to program the TI Microcontrollers easily.

The main aim of Energia is to make programming TI MCU's as easy as programming in Arduino.

Energia is an Equivalent for Arduino that supports Texas Instruments Microcontrollers.

The MSP430 Development Ecosystem

Code Composer Studio (CCS) is a more versatile **professional IDE** which has more functionalities and capabilities in terms of accessing the inside architecture of microcontroller.

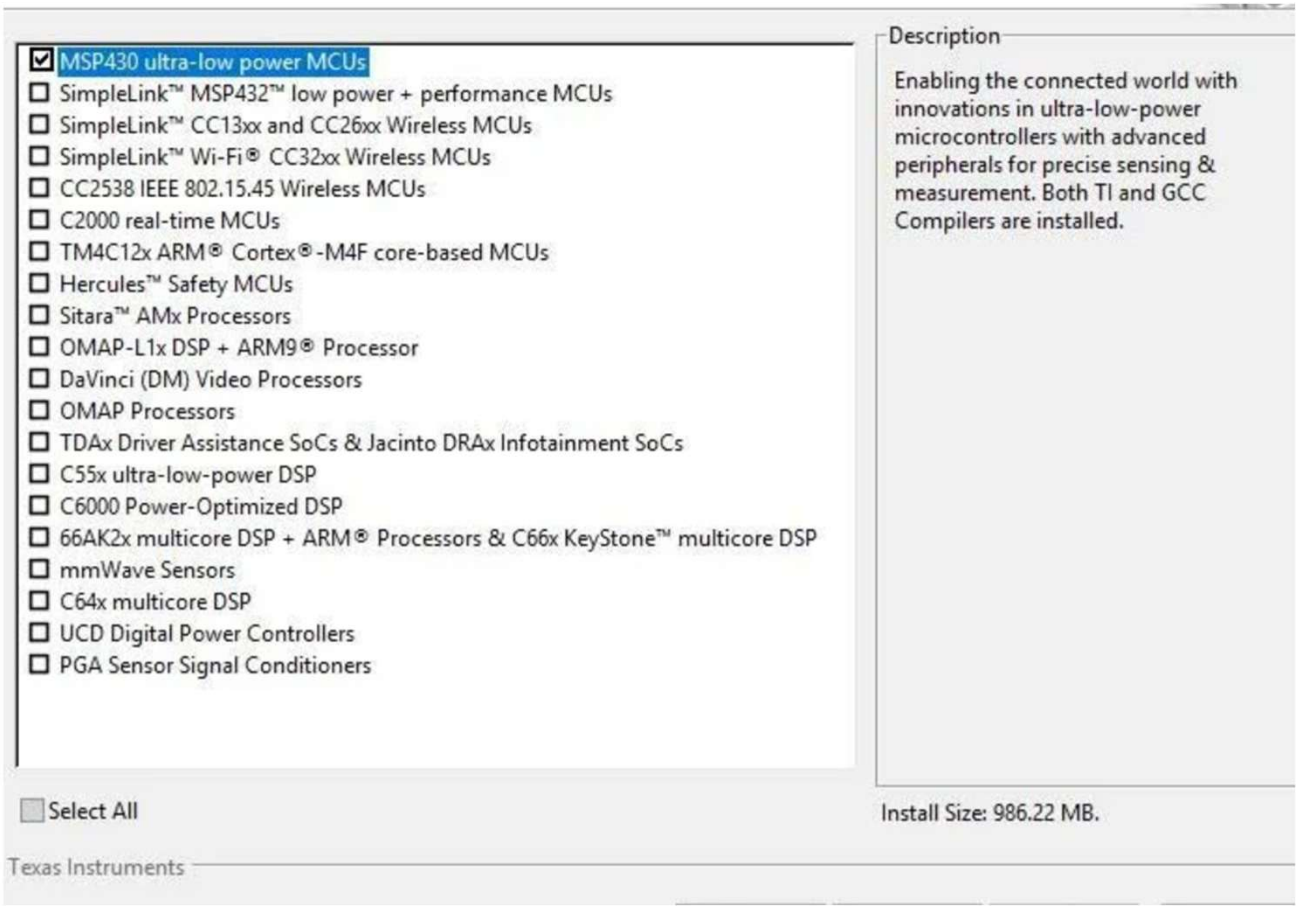
It has inbuilt debugging function which can check errors in your code and you can run your code line by line that helps in finding error without any headache.

"It will take sometime to get comfortable with CCS. Once you set with this awesome software, trust me you will get to know anything about a particular microcontroller.

You have to take help of Datasheet of the microcontroller to write your program"

https://circuitdigest.com/microcontroller-projects/getting-started-with-msp430-using-code-composer-studio

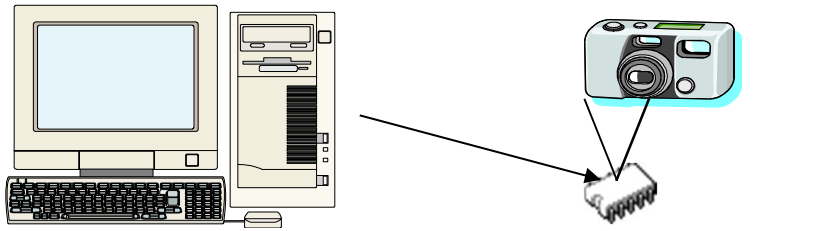
The MSP430 Development Ecosystem



Assembly and C examples are available, may have to be downloaded the first time

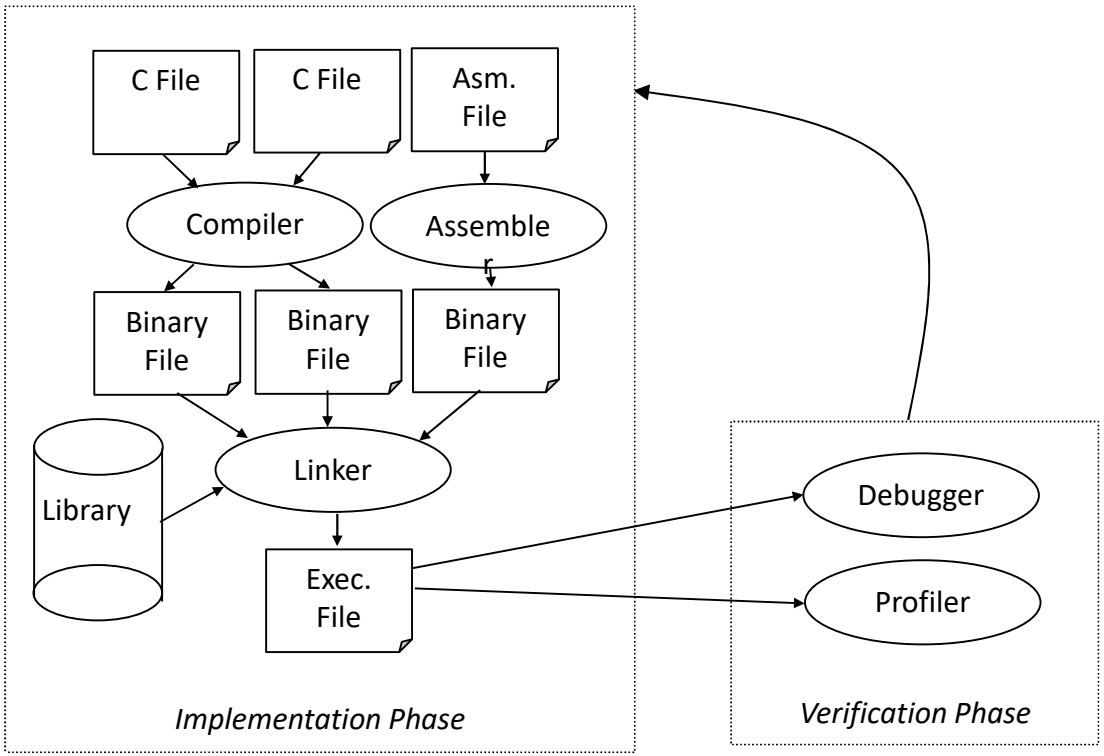
Development Environment

- Development processor
  - The processor on which we write and debug our programs
    - Usually a PC
- *Target processor*
  - The processor that the program will run on in our embedded system
    - Often different from the development processor



Development processor Target processor

Software Development Process



- Compilers
  - Cross compiler
    - Runs on one processor, but generates code for another
- Assemblers
- Linkers
- Debuggers
- Profilers

Running a Program

- If development processor is different than target, how can we run our compiled code? Two options:
  - Download to target processor
  - Simulate
- Simulation
  - One method: Hardware description language
    - But slow, not always available
  - Another method: *Instruction set simulator (ISS)*
    - Runs on development processor, but executes instructions of target processor

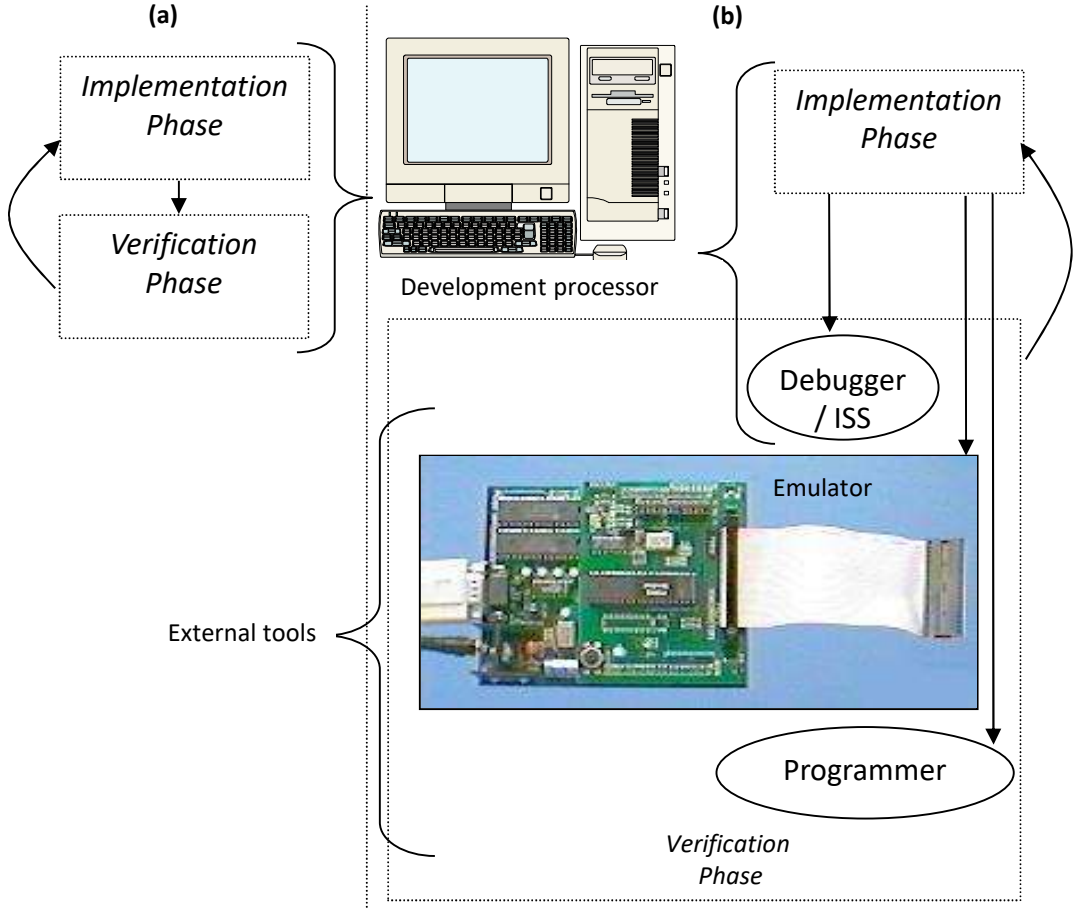
PES UNIVERSITY

Running a Program

- ISS
  - Gives us control over time – set breakpoints, look at register values, set values, step-by-step execution, ...
  - But, doesn't interact with real environment
- Download to board
  - Use device programmer
  - Runs in real environment, but not controllable
- Compromise: emulator
  - Runs in real environment, at speed or near
  - Supports some controllability from the PC

PES UNIVERSITY

Testing and Debugging



Emulators can be an external hardware between development processor and target, or part of target board

Laboratory Component

i)Getting started with IDE:
ii)Introduction to Code Composer Studio and IAR Workbench

Programming

i) Data Movement Programs
ii) Assembly programs using different directives
iii) MOV instruction and addressing modes
iv) Subroutine calls and use store values to Stack and restore values
v) Memory Map of MSP430G2553

Arithmetic and Logical Instructions:
ADD, ADC, AND, DEC, INC, SUB, CALL, RET, RLA, RLC, RRA, RRC etc.

Laboratory Component

Peripherals: Timers and interrupts
i)Basics to Digital IO and Timer
ii)to use Timers for generating the needed delays
iii)program for generating interrupts for timer and digital inputs

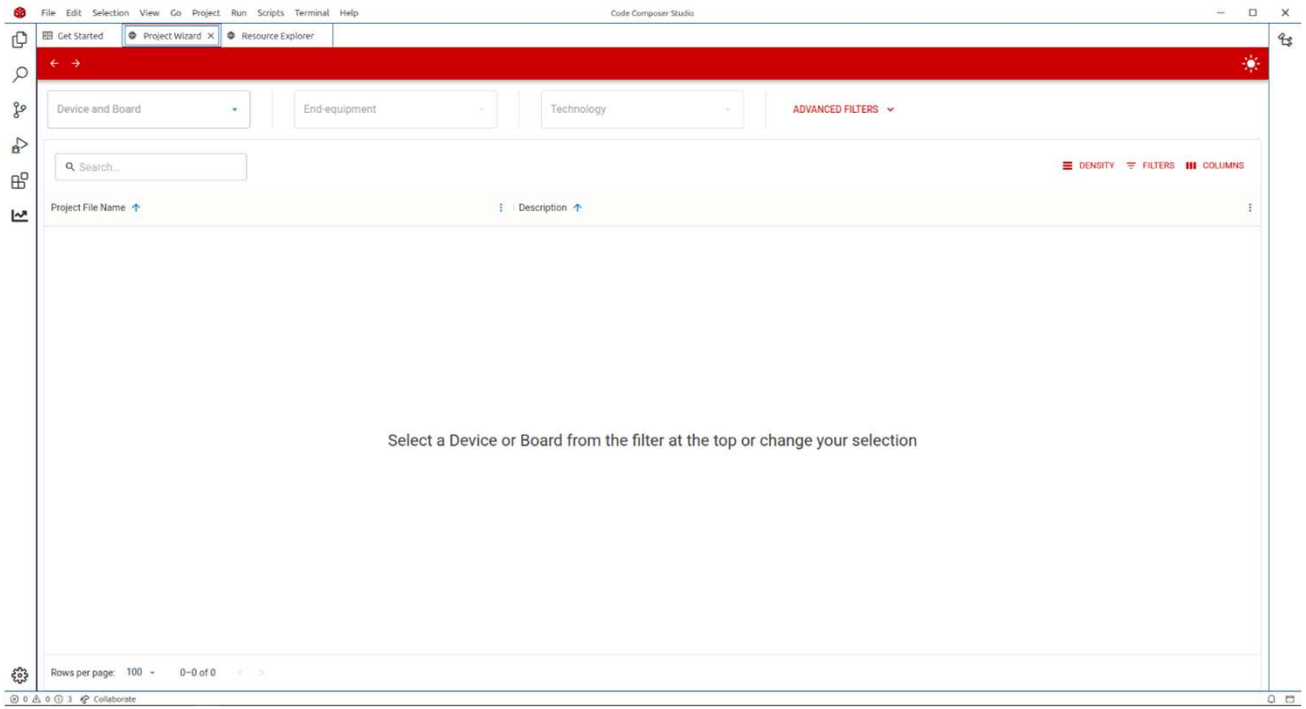
Applications
i) The hex key pad driver will give user inputs
ii) The user outputs are displayed on LCD.
Servo motor control, Stepper motor control
DCMotor control
i)Controlling direction of motor
ii)Controlling Speed of motor
iii) Generating pulse width modulation

** You will do the lab only if you know what to do! Come prepared

Lab Exercise 1

Blink onboard LED

Using the example files in the CCS IDE



Processor is MSP430G2553

Lab Exercise 1

```
47   //  E. Chen
48   //  Texas Instruments, Inc
49   //  May 2018
50   //  Built with CCS Version 8.0 and IAR Embedded Workbench Version: 7.11
51   //******************************************************************************
52
53   #include <msp430.h>
54
55   int main(void)
56   {
57       volatile unsigned int i;
58       WDTCTL = WDTPW + WDTHOLD;                 // Stop watchdog timer
59       P1DIR |= 0x01;                            // Set P1.0 to output direction
60
61       while(1)
62       {
63           P1OUT ^= 0x01;                        // Toggle P1.0 using exclusive-OR
64
65           for (i=10000; i>0; i--);
66       }
67   }
68
```

Problems 3 | Output | Debug Output | Debug Console ×

Lab Exercise 1 - C

```
47  //  E. Chen
48  //  Texas Instruments, Inc
49  //  May 2018
50  //  Built with CCS Version 8.0 and IAR Embedded Workbench Version: 7.11
51  //******************************************************************************
52
53  #include <msp430.h>
54
55  int main(void)
56  {
57      volatile unsigned int i;
58      WDTCTL = WDTPW + WDTHOLD;                 // Stop watchdog timer
59      P1DIR |= 0x01;                            // Set P1.0 to output direction
60
61      while(1)
62      {
63          P1OUT ^= 0x01;                        // Toggle P1.0 using exclusive-OR
64
65          for (i=10000; i>0; i--);
66      }
67  }
68
```

Infinite loop, typical

How long is the delay? Clock frequency?
Why decrement rather than increment?

Problems 3 | Output | Debug Output | Debug Console

Lab Exercise 1 - Assembly

```
;   D. Dang
;   Texas Instruments Inc.
;   December 2010
;   Built with Code Composer Essentials Version: 4.2.0
;-------------------------------------------------------------------------------
.cdecls C,LIST,  "msp430.h"
;-------------------------------------------------------------------------------
            .def    RESET                   ; Export program entry-point to
                                            ; make it known to linker.

;-------------------------------------------------------------------------------
            .text                           ; Progam Start
;-------------------------------------------------------------------------------
RESET       mov.w   #0280h,SP               ; Initialize stackpointer
StopWDT     mov.w   #WDTPW+WDTHOLD,&WDTCTL  ; Stop WDT
SetupP1     bis.b   #001h,&P1DIR            ; P1.0  output
            ;
Mainloop    xor.b   #001h,&P1OUT            ; Toggle P1.0
Wait        mov.w■    #050000,R15             ; Delay to R15
L1          dec.w   R15                     ; Decrement R15
            jnz     L1                      ; Delay over?
            jmp     Mainloop                ; Again
            ;
;-------------------------------------------------------------------------------
;           Interrupt Vectors
;-------------------------------------------------------------------------------
            .sect   ".reset"                ; MSP430 RESET Vector
            .short  RESET                   ;
            .end
```

A bit more daunting!

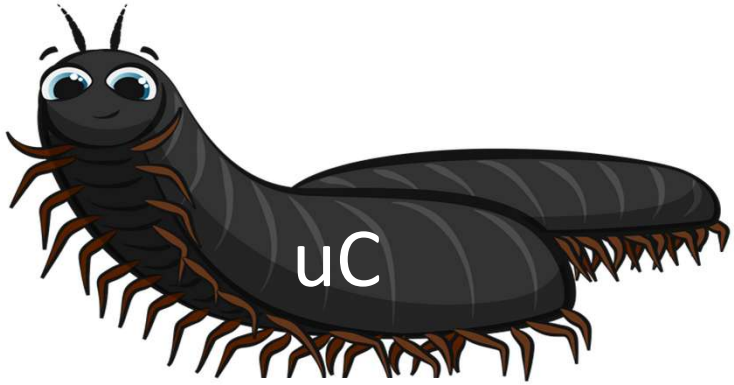
.h file not mandatory
Can refer to ports and registers directly
Another lab task!

All of them have addresses

Different ‘addressing modes’
Instruction set

PES UNIVERSITY

Course ‘outcomes’



Will it bite?

Not if you are the master

Adequate preparation, relearn **applications**

Lab and theory to reinforce

THANK YOU

Rex Joseph

Department of Electrical and Electronics Engineering

rexjoseph@pes.edu

+91 80 6666 3333 Extn 515